Migration breakdown: `20260805000000_trades_idempotency_guard.sql`

Source file: `supabase/migrations/20260805000000_trades_idempotency_guard.sql` **Status:** NOT yet applied on prod (`sxkpcovbyaficvtkpsdo`). Present on `upstream/dev`, `upstream/staging`, `origin/main`; MISSING on `upstream/main`.

This document explains, statement by statement, what the SQL does and why. It changes no data. It only creates indexes, adds one comment, and registers the migration. If anything fails, the whole thing rolls back and nothing changes.

The problem it solves (plain English)

The worker "claim" prevents the same signal from being dispatched twice. But claims are stored, and a worker can crash, retry, or lose its own response after it already sent an order to the broker. In that window the same broker order ticket could be written to the `trades` table twice for the same broker account - one real order showing up as two rows.

The worker logic reduces the chance of this. This migration makes the database itself refuse it: a uniqueness constraint that is impossible to bypass, no matter what the worker does.

The guard only applies to trades created from **2026-08-05** (the install date) onward. Older rows are left untouched on purpose, so the index can be built on top of existing data without deleting or merging anything.

Statement 1 - the pre-flight duplicate check

```
DO $$
DECLARE
  duplicate_count bigint;
  cutoff timestamptz := '2026-08-05T00:00:00Z';
BEGIN
  SELECT count(*)
    INTO duplicate_count
  FROM (
    SELECT broker_account_id, metaapi_order_id
    FROM public.trades
    WHERE broker_account_id IS NOT NULL
      AND metaapi_order_id IS NOT NULL
      AND created_at >= cutoff
    GROUP BY broker_account_id, metaapi_order_id
    HAVING count(*) > 1
  ) duplicates;

  IF duplicate_count > 0 THEN
    RAISE EXCEPTION
      'Cannot create trades broker-ticket uniqueness guard: % duplicate broker ticket groups created after the cutoff require manual audit first',
      duplicate_count
      USING HINT = 'Group trades by broker_account_id and metaapi_order_id, reconcile the broker records, then rerun this migration.';
  END IF;
END $$;
```

A `DO $$ ... $$` block is an anonymous, one-off block of PL/pgSQL. It runs immediately and leaves nothing behind.

What each piece does:

- **`duplicate_count bigint`** - a variable that will hold the number of duplicate groups found.
- **`cutoff timestamptz := '2026-08-05T00:00:00Z'`** - a variable holding the cutoff timestamp. The letter `Z` means UTC.
- **`SELECT count(*) ... FROM (...)`** - counts how many groups of duplicates exist.
- **The inner query** - picks rows where **both** `broker_account_id` and `metaapi_order_id` are filled in, and the trade was created at/after the cutoff. It then groups rows by that pair and keeps only groups where the pair appears more than once (`HAVING count(*) > 1`).
- **`IF duplicate_count > 0 THEN RAISE EXCEPTION ...`** - if any duplicate group exists, the migration stops immediately and prints an error telling you how many groups need auditing.

Why this check exists: a unique index cannot be created while duplicate rows already exist in the range it covers - the `CREATE UNIQUE INDEX` would fail on its own. This check turns that confusing failure into a clear, actionable message, and it happens *before* any index is touched.

Important: this block only *reads* data. It never modifies anything. On a re-run it does the same read again and either passes (no duplicates) or aborts (duplicates still there).

Statement 2 - the actual guard: a partial unique index

```
CREATE UNIQUE INDEX IF NOT EXISTS trades_broker_order_unique_idx
  ON public.trades (broker_account_id, metaapi_order_id)
  WHERE broker_account_id IS NOT NULL
    AND metaapi_order_id IS NOT NULL
    AND created_at >= '2026-08-05T00:00:00Z';
```

What each piece means:

- **`CREATE UNIQUE INDEX`** - the database will refuse to store two rows with the same index values. This is the enforcement the migration is named after.
- **`IF NOT EXISTS`** - if the index already exists, this statement does nothing instead of failing. This is what makes the migration safe to run more than once.
- **`trades_broker_order_unique_idx`** - the name of the index, chosen so it can be dropped or inspected later.
- **`ON public.trades (broker_account_id, metaapi_order_id)`** - the uniqueness key: one broker account + one broker order ticket.
- **`WHERE broker_account_id IS NOT NULL AND metaapi_order_id IS NOT NULL`** - this is a *partial* index. It only applies to rows that actually have a real broker ticket. Rows with no ticket yet (open trades awaiting a fill) are excluded, so they never block anything.
- **`AND created_at >= '2026-08-05T00:00:00Z'`** - the partial index also only covers trades from the cutoff onward. Historical duplicate tickets (pre-cutoff) are outside its scope, so they don't stop the index from being created.

Result: from the cutoff onward, one broker ticket can appear at most once per broker account. A second insert of the same ticket is rejected by the database with a unique-violation error - that is the failure mode the worker now catches instead of silently writing a duplicate row.

Statement 3 - a supporting read index (not a constraint)

```
CREATE INDEX IF NOT EXISTS trades_signal_broker_opened_idx
  ON public.trades (signal_id, broker_account_id, opened_at)
  WHERE signal_id IS NOT NULL
    AND broker_account_id IS NOT NULL;
```

What each piece means:

- **`CREATE INDEX (no UNIQUE)`** - a normal, non-unique index. It enforces nothing; it only speeds up lookups.
- **`IF NOT EXISTS`** - same idempotency as above - a re-run does nothing.
- **`trades_signal_broker_opened_idx`** - the index name.
- **`ON public.trades (signal_id, broker_account_id, opened_at)`** - the columns being indexed, in order: which signal, which broker account, when the trade opened.
- **`WHERE signal_id IS NOT NULL AND broker_account_id IS NOT NULL`** - a partial index again - only rows that are fully linked to a signal and a broker account are indexed.

Why it exists: queries that look up trades by signal + broker account + open time (for example the review/audit flows and reconciliation) can use this index instead of scanning the whole table.

Statement 4 - a comment on the index

```
COMMENT ON INDEX public.trades_broker_order_unique_idx IS
  'Prevents the same broker order ticket from being persisted twice for one broker account (applies to trades created from 2026-08-05 onward; earlier rows are excluded from the uniqueness check).';
```

`COMMENT ON INDEX` attaches a human-readable description to the index. It stores no data and changes no behavior. Its only purpose is that anyone inspecting the database later (in the Supabase dashboard, `pg_indexes`, or a query) immediately sees *why* the index has that unusual partial `WHERE` clause.

Statement 5 - registering the migration as applied

```
INSERT INTO supabase_migrations.schema_migrations (version, statements, name)
VALUES ('20260805000000', '{ "-- migration applied via API" }', '20260805000000_trades_idempotency_guard.sql')
ON CONFLICT (version) DO NOTHING;
```

What each piece means:

- **`INSERT INTO supabase_migrations.schema_migrations`** - records in Supabase's migration-tracking table that this migration was applied.
- **`version`** - the migration's version string, `20260805000000`, matching the file name.
- **`statements`** - a placeholder note saying it was applied via the API (not via the CLI).
- **`name`** - the migration file name.
- **`ON CONFLICT (version) DO NOTHING`** - if this version is already recorded, the insert is skipped. This keeps the whole migration idempotent.

Why it matters: Supabase tooling decides what still needs to be applied by comparing its migration files against this table. Without the registration row, the next `supabase db push` would think this migration was never applied and try to run it again. With it, tooling knows it is already done.

Why this migration is safe

1. **No data is changed.** There is no `UPDATE`, `DELETE`, `DROP`, or `ALTER` anywhere in the file.
2. **Every statement is idempotent.** The two indexes use `IF NOT EXISTS`, the registration uses `ON CONFLICT ... DO NOTHING`, and the `DO` block only reads. Running the file twice is the same as running it once.
3. **It aborts cleanly if it cannot proceed.** If duplicate broker tickets exist at/after the cutoff, the `DO` block raises an error, the transaction rolls back, and not even the indexes are created. Nothing is left half-done.
4. **Historical data is untouched.** Pre-cutoff duplicate tickets are excluded from the index scope, so the migration never has to delete or merge them. They still need a separate manual audit - the migration's error message says so explicitly when it detects post-cutoff duplicates.

The one real side effect

Creating the unique index takes a short lock on the `trades` table while it is built. During that brief window, concurrent writes to `trades` wait. This is expected for a one-time migration on a table that is actively written to, and it is not data loss.

Full SQL for reference

```
DO $$
DECLARE
  duplicate_count bigint;
  cutoff timestamptz := '2026-08-05T00:00:00Z';
BEGIN
  SELECT count(*)
    INTO duplicate_count
  FROM (
    SELECT broker_account_id, metaapi_order_id
    FROM public.trades
    WHERE broker_account_id IS NOT NULL
      AND metaapi_order_id IS NOT NULL
      AND created_at >= cutoff
    GROUP BY broker_account_id, metaapi_order_id
    HAVING count(*) > 1
  ) duplicates;

  IF duplicate_count > 0 THEN
    RAISE EXCEPTION
      'Cannot create trades broker-ticket uniqueness guard: % duplicate broker ticket groups created after the cutoff require manual audit first',
      duplicate_count
      USING HINT = 'Group trades by broker_account_id and metaapi_order_id, reconcile the broker records, then rerun this migration.';
  END IF;
END $$;

CREATE UNIQUE INDEX IF NOT EXISTS trades_broker_order_unique_idx
  ON public.trades (broker_account_id, metaapi_order_id)
  WHERE broker_account_id IS NOT NULL
    AND metaapi_order_id IS NOT NULL
    AND created_at >= '2026-08-05T00:00:00Z';

CREATE INDEX IF NOT EXISTS trades_signal_broker_opened_idx
  ON public.trades (signal_id, broker_account_id, opened_at)
  WHERE signal_id IS NOT NULL
    AND broker_account_id IS NOT NULL;

COMMENT ON INDEX public.trades_broker_order_unique_idx IS
  'Prevents the same broker order ticket from being persisted twice for one broker account (applies to trades created from 2026-08-05 onward; earlier rows are excluded from the uniqueness check).';

INSERT INTO supabase_migrations.schema_migrations (version, statements, name)
VALUES ('20260805000000', '{ "-- migration applied via API" }', '20260805000000_trades_idempotency_guard.sql')
ON CONFLICT (version) DO NOTHING;
```